SMS++ File Format Reference

Version: 0.50

Date: 27/03/2019

Contributors:	Name	Organisation
Deliverable Leader	Antonio Frangioni	ICOOR (UniPi)
Work Package Leader	Utz-Uwe Haus	Cray
Contributing Authors	Antonio Frangioni Laura Galli Rafael Durbano Lobato Ali Ghezelsoflu Niccolò Iardella	ICOOR (UniPi)

Contents

1 Introduction

This document describes the format of input files for the SMS++ components of the PLAN4RES project. These are the components tasked with performing the actual solution of the different optimization problems involved in the process. The production of these files possibly requires:

- a sophisticated data extraction and preparation phase;
- a tuning phase aimed at finding appropriate parameters for the different modelling and algorithmic options of the solution process.

The description of both phases is outside of the scope of this document, which only deals with the required final form of the input.

Insomuch as the set of types of optimization problems (`Block` in SMS++ parlance) to be solved within the project, and the set of available solution methods for doing it (`Solver` in SMS++ parlance) may be expanding over time, this document is intended as dynamic: as new components will added, the document will grow to cover the corresponding input formats. Non-backward-compatible modification of existing input formats will be kept to a minimum, and always agreed upon with the involved partners before being put into production. Appropriate versioning of both the software and this document will ensure consistency between the data preparation and data consumption processes.

The self-documenting nature of NETCDF, together with its portable readability and the schema definition of the various `Block`, will make it possible to re-use all or some blocks of one SMS++-generated model in other contexts.

2 General netCDF file structure

NETCDF (network Common Data Form) [1] is a set of interfaces for array-oriented data access and a freely distributed collection of data access libraries for C, Fortran, C++, Java, and other languages. The NETCDF libraries support a machine-independent format for representing scientific data. Together, the interfaces, libraries, and format support the creation, access, and sharing of scientific data.

NETCDF data is:

- Self-Describing. A NETCDF file includes information about the data it contains.
- Portable. A NETCDF file can be accessed by computers with different ways of storing integers, characters, and floating-point numbers.
- Scalable. A small subset of a large dataset may be accessed efficiently.
- Appendable. Data may be appended to a properly structured NETCDF file without copying the dataset or redefining its structure.
- Sharable. One writer and multiple readers may simultaneously access the same NETCDF file.
- Archivable. Access to all earlier forms of NETCDF data will be supported by current and future versions of the software.

For the purpose of this document, only a relatively small set of NETCDF concepts need be discussed.

- NETCDF *file*: is the basic container of NETCDF, e.g. corresponding to a standard file in the filesystem of the host machine. In the C++ interface used by SMS++ it corresponds to an object of type `netCDF::NcFile`. A typical constructor of the object requires the pathname of the file (a standard `char *`) and the mode, which can be e.g. `netCDF::NcFile::read` or `netCDF::NcFile::replace` for reading and writing, respectively.
- NETCDF *group*: a NETCDF file is organised into several *groups*, which can be nested inside each other to an arbitrary degree. In the C++ interface used by SMS++, each group corresponds to an object of type `netCDF::NcGroup`; note that `netCDF::NcFile` derives from `netCDF::NcGroup`, and therefore all capabilities of the group are shared by the file (but not vice-versa). Each *group* has a unique string name by which it can retrieved; `netCDF::NcGroup` (and, therefore, `netCDF::NcFile`) has a method `getGroup()` taking as input a `std::string` with the group name and returning an `netCDF::NcGroup` object containing the *child group* with the given name (if any) of that group.
- NETCDF *attribute*: is a simple data point specific of a netCDF group, having a unique string name by which it can retrieved and a simple value (`int`, `double`, `std::string`, ...). In the C++ interface used by SMS++, it is represented by a `netCDF::NcGroupAtt` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getAtt()` taking in input its name. Then, the method(s) `getValue()` (available with different signatures corresponding to the different supported basic data types) allows to retrieve its value.
- NETCDF *dimension*: it basically represent the different indices over which the actual data is indexed, each of which is given a name. In the C++ interface used by SMS++, it is represented by a `netCDF::NcDim` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getDim()` taking in input its name. The dimension (a `size_t` value) is returned by the method `getSize()`.
- NETCDF *variable*: this is the container for the "complex" data of a *groups*. It has a name (`std::string`), a basic data type (`int`, `double`, `std::string`, ...) and in principle any number of *dimensions*. In the C++ interface used by SMS++, it is represented by a `netCDF::NcVar` object, which can be obtained by its enclosing `netCDF::NcGroup` via the method `getVar()` taking in input its name. Once the `netCDF::NcVar` object is obtained, individual values of the *variable* can be read by its method(s) `getVar()` (available with different signatures corresponding to the different supported basic data types), taking a `std::vector<size_t>` as long as the the number of *dimensions* of the *variable* (obtainable e.g. with its method `NgetDimCount()`).

The above components allow to read and write structured data files with basically any nested format, which makes NETCDF ideal for representing the data characterizing SMS++, as discussed below.

3 General SMS++ file structure

In SMS++, files can describe two different kinds of objects:

1. `Block` objects, representing the data characterizing a specific instance of a (structured) optimization problem to be solved;

2. `Configuration` objects, representing parameters of the solution process, either pertaining to a `Block` (say, which of the different available formulations of the optimization problem should be used), or to a `Solver` used to actually perform the optimizaton process.

Both objects have `serialize()` and `deserialize()` methods allowing to save the current state to a given NETCDF *file*/*group*, as well as *factories* allowing to create a new object from scratch out of a given NETCDF *file*/*group*.

The issue of SMS++ is that an optimization problem is represented by a `Block`, which can have any number of sub-`Block` (recursively) representing parts of the problem with a specific structure. Yet, "specific structure" here may mean that the enclosing `Block` can make some assumptions on what kind of sub-`Block` it contains, but it may not know exactly their type. In C++ parlance, `Block` is an abstract base class, and specific optimization problems are represented by specific derived classes (`:Block`). A given `:Block` can make assumption on the type of its sub-`Block`, but there can be many different variants of it, represented by different derived classes from the same base one. Thus, clearly the data-reading must be organized in such a way that each `:Block` is responsible for retrieving that which is necessary to it without making assumptions about the global organization of the file. This does not only apply to the data of the instance, but also to *configuration* options that a `:Block` may have. In fact, several aspect of the `:Block` representing a given problem (which formulation is used, if and how variables and constraint of the formulation are dynamically generated, which parts of the primal/dual solution are saved, ...) can be implemented in different ways. Note that these decisions are in principle independent from the specific instance, and therefore are conceptually to be stored in, and retrieved from, a separate container than that holding the data of the `Block`; this is why the `Configuration` class is provided. Since `Block` is a tree-shaped nested data structure, `Configuration` also have to be such. Clearly, NETCDF perfectly suit this kind of requirement; in particular, the standing assumption for both classes is that

the data of one object (`Block`/`Configuration`) *is all to be found in the same* NETCDF *group;*
the data of its "sub-objects" (sub-`Block`*/sub-*`Configuration`*) is to be found in child groups of the group.*

Hence, the format of a SMS++ NETCDF file depends on wether it holds `Block` information, `Configuration` information, or both. For the latter, there are actually two separate kind of `Configuration` information that can be attached to a `:Block`. The first describes the above-mentioned options that are inherent to the `:Block` itself, which are described by the specific derived class `BlockConfig` of `Configuration`. However, a `:Block` need be solved, which is done by `Solver` objects; each `Block` can have an arbitrary number of `Solver` "registered" with it. A `Solver` can be an arbitrarily complex solution process; in particular, the structure of `Block` immediately implies that one can register `Solver` with any sub-`Block`, which is actually useful e.g. to decomposition algorithms. Hence, not only each `:Solver` can have an arbitrarily large set of algorithmic configuration options; an arbitrarily large set of `Solver` can be attached to a `Block` and to all of its sub-`Block`, recursively. Thus, the different derived class `BlockSolverConfig` of `Configuration` is provided to describe all this information. Note, again, that this is conceptually separate from both the specific instance encoded in the `:Block` and the choice of the `Block`-specific options encoded in the `BlockConfig`, which justifies why a separate container is used.

To allow holding all this different information, a SMS++ NETCDF file has the following general structure. It must contain (in the `netCDF::NcFile` object, which is also a `netCDF::NcGroup`) an *attribute* "`SMS++_file_type`" of `int` type which specifies which of three different kinds of file it is. The values are encoded in the `enum smspp_netCDF_file_type` defined in `SMSTypedefs.h`, as follows:

- `eProbFile = 0`: the *file* (which is also a *group*) has any number of child *groups* with names "`Prob_0`", "`Prob_1`", ... In turn, each child *group* has *exactly three* child *groups* with names "`Block`", "`BlockConfig`" and "`BlockSolver`", respectively. The first is intended to contain the serialization of a `:Block`, the second the serialization of a `:BlockConfig` of the same `:Block`, and the third the serialization of a `:BlockSolverConfig` of the same `:Block`, although any of the three can in principle be empty. If any of the child is not empty, it *must* necessarily contain all the information necessary to reconstruct the specific instance.

- `eBlockFile = 1`: the *file* (which is also a *group*) has any number of child *groups* with names "`Block_0`", "`Block_1`", ... Each child *group* contains the serialization of a `:Block`.
- `eConfigFile = 2`: the *file* (which is also a *group*) has any number of child *groups* with names "`Config_0`", "`Config_1`", ... Each child *group* contains the serialization of a `:Configuration`. Note that no distinction is required between a `:BlockConfig` and a `:BlockSolverConfig`, since they are both derived from `Configuration`.

Note that (some variants of) the `deserialize()` methods of `Block`/ `Configuration` allow to directly specify which of the different objects stored in the file to read via its index i in the "name".

To completely describe the file formats we now have to describe the format of the NETCDF *group* as far as the base `Block` and `Configuration` (for the latter, actually both sub-classes `BlockConfig` and `BlockSolverConfig`) are concerned, i.e., the (possibly, small) part of the contents that do not depend on the specific derived class.

3.1 `Block` description

The only content that any NETCDF *group* describing a `:Block` must necessarily have is a string *attribute* with name "`type`" containing the classname of the object. This must be exact, e.g., as returned by the protected virtual method `name()`, since it is used in the *factory* when deserializing the object. Note that template classes derived from `Block` are allowed, with the standard notation "`class_name<template_paramaters>`".

3.2 `Configuration` description

The only content that any NETCDF *group* describing a `:Configuration` must necessarily have is a string *attribute* with name "`type`" containing the classname of the object. This must be exact, e.g., as returned by the protected virtual method `name()`, since it is used in the *factory* when deserializing the object. Note that template classes derived from `Configuration` are allowed, with the standard notation "`class_name<template_paramaters>`"; see for instance the template `SimpleConfiguration<>` class.

3.2.1 `BlockConfig` description

Besides the mandatory "`type`" attribute of any `:Configuration`, a SMS++ NETCDF *group* for a `BlockConfig` can contain the following:

- the *attribute* "`name`" of string type containing the "name" of the `Block` (this is not a classname, but rather a string describing something about the `Block` that is supposedly useful when printing/debugging);
- the *group* "`static_constraints`" containing a `Configuration` object for the static constraints of the `Block`;
- the *group* "`dynamic_constraints `" containing a `Configuration` object for the dynamic constraints of the `Block`;
- the *group* "`static_variables`" containing a `Configuration` object for the static variables of the `Block`;
- the *group* "`dynamic_variables`" containing a `Configuration` object for the dynamic variables of the `Block`;
- the *group* "`objective`" containing a `Configuration` object for the objective of the `Block`;
- the *group* "`is_feasible`" containing a `Configuration` object for the `is_feasible()` method of the `Block`;
- the *group* "`is_optimal`" containing a `Configuration` object for the `is_optimal()` method of the `Block`;
- the *group* "`solution`" containing a `Configuration` object for the methods of the `Block` dealing with solutions (`get_Solution()` and `map_[back/forward]_solution()`);
- the *group* "`extra`" containing a `Configuration` object, which has no direct use in the base `Block` class, but is added so that derived classes can put there any configuration information without having to define further derived classes form `BlockConfig` (which, however, they can still do if they want);

- the *dimension* "n_sub_Block" containing the number of BlockConfig descriptions for the sub-Block of the current :Block;
- with n being the size of n_sub_Block, n *groups*, with name "sub-BlockConfig_i" for all $i = 0, \ldots, n-1$, containing each the description of a BlockConfig for one of the sub-Block of the current :Block.

Of all these, only the "name" *attribute* and the "n_sub_Block" *dimension* are mandatory (they must exist, although they may be empty/zero). All other groups may not exist, in which case the corresponding field of the class is filled with a nullptr, indicating that the "default" configuration (whatever that may mean for the :Block in question) have to be used. Note that that the matching between the sub-BlockConfig and the sub-Block is positional: the BlockConfig found in the group "sub-BlockConfig_i" is that for the i-th sub-Block. Note that the vector of sub-BlockConfig is allowed to be of different size than the number of sub-Block; if it is larger any extra BlockConfig is simply ignored, if it is shorter then all missing sub-BlockConfig are treated as nullptr (default configuration).

3.2.2 BlockSolverConfig description

Besides the mandatory "type" attribute of any :Configuration, a SMS++ NETCDF *group* for a BlockSolverConfig can contain the following:

- the *dimension* "n_SolverConfig" containing the number of Solver that are to be attached to this Block, and therefore the (expected) number of their :Configuration objects;
- the *variable* "SolverNames", of type string and indexed over the dimension "n_SolverConfig"; the i-th entry of the variable is assumed to contain the classname of a :Solver object to be attached to the Block (this must be exact, e.g., as returned by the protected virtual method name(), since it is used in the *factory* when creating the object);
- with n being the size of n_SolverConfig, n *groups*, with name "SolverConfig_i" for all $i = 0, \ldots, n-1$, containing each the description of a :Configuration object for the i-th :Solver;
- the *dimension* "n_BlockSolverConfig" containing containing the number of BlockSolverConfig descriptions for the sub-Block of the current :Block;
- with m being the size of n_BlockSolverConfig, m *groups*, with name "BlockSolverConfig_i" for all $i = 0, \ldots, m-1$, containing each the description of a BlockSolverConfig for one of the sub-Block of the current :Block.

Of all these, only the "n_SolverConfig" and "n_BlockSolverConfig" *dimensions* and the "SolverNames" *variable* are mandatory; the individual *groups* may not exist if the corresponding dimension is 0. Note that that the matching between the sub-BlockSolverConfig and the sub-Block is positional: the BlockSolverConfig found in the group "BlockSolverConfig_i" is that for the i-th sub-Block. Note that the vector of sub-BlockSolverConfig is allowed to be of different size than the number of sub-Block; if it is larger any extra BlockSolverConfig is simply ignored, if it is shorter then all missing sub-BlockConfig are treated as nullptr (default configuration).

Note that the :Configuration objects to be found in the *groups* "SolverConfig_i" are not "any" :Configuration; rather, they are assumed to be of the specific type ComputeConfig, whose format is described in the next subsection.

3.2.3 ComputeConfig description

ComputeConfig (defined in ThinComputeInterface.h) is a class of :Configuration objects specifically tailored for representing sets of algorithmic parameters of solution algorithms, like the ones presumably required by a :Solver. Besides the mandatory "type" attribute of any :Configuration, a SMS++ NETCDF *group* for a ComputeConfig can contain the following:

- the *attribute* "diff" of int type containing the value for the f_diff field of the ComputeConfig (basically, a bool telling if the information in it has to be taken as "the configuration to be set" or as "the changes to be made from the current configuration");
- the *dimension* "num_int_par" containing the number of int parameters;

- the *variable* "int_par_names", of type string and indexed over the dimension "num_int_par"; the i-th entry of the variable is assumed to contain the string name of an int parameter;
- the *variable* "int_par_vals", of type int and indexed over the dimension "num_int_par"; the i-th entry of the variable is assumed to contain the value of the int parameter whose string name is to be found in the i-th entry of "int_par_names";
- the *dimension* "num_dbl_par" containing the number of double parameters;
- the *variable* "dbl_par_names", of type string and indexed over the dimension "num_dbl_par"; the i-th entry of the variable is assumed to contain the string name of a double parameter;
- the *variable* "dbl_par_vals", of type double and indexed over the dimension "num_dbl_par"; the i-th entry of the variable is assumed to contain the value of the double parameter whose string name is to be found in the i-th entry of "dbl_par_names";
- the *dimension* "num_str_par" containing the number of string parameters;
- the *variable* "str_par_names", of type string and indexed over the dimension "num_str_par"; the i-th entry of the variable is assumed to contain the string name of a string parameter;
- the *variable* "str_par_vals", of type string and indexed over the dimension "num_str_par"; the i-th entry of the variable is assumed to contain the value of the string parameter whose string name is to be found in the i-th entry of "str_par_names";
- the *group* "extra" containing a Configuration object, which has no direct use in the base ComputeConfig class, but is added so that derived classes can put there any configuration information without having to define further derived classes form ComputeConfig (which, however, they can still do if they want).

The three *dimensions* "num_*_par" are mandatory. If any of these is zero, the corresponding variables "*_par_names" and "*_par_vals" may not be defined. The "extra" *group* may not exist, in which case the corresponding Configuration object is set to a nullptr.

4 File formats of existing :Block

This Section collects the description of the NETCDF file formats for all the implemented :Block developed for PLAN4RES. As such, the section will be continuously updated over the course of the project as new :Block will be added, although as usual non-backward-compatible changes to existing input formats will be kept to a minimum, and always agreed upon with the involved partners before being put into production.

4.1 SimpleMILPBlock

SimpleMILPBlock is a simple classe, only having the "abstract representation", for describing Mixed-Integer Linear Programs of the generic form

$$\min\{\, \bar{c} + cx \,:\, \underline{b} \leq Ax \leq \bar{b} \;\;,\;\; l \leq x \leq u \,\}$$

where c, l, $u \in \mathbb{R}^n$, $\underline{b}$, $\bar{b} \in \mathbb{R}^m$, $A \in \mathbb{R}^{m \times n}$, and $c^0 \in \mathbb{R}$. Actually, entries of $\underline{b}$ and l can be $-\infty$, and entries of $\bar{b}$ and u can be $+\infty$. The coefficient matrix A is usually very sparse, which justifies the choice of a standard sparse (row-wise) format to represent it.

Besides the mandatory "type" *attribute* of any :Block, the group will contain the following:

- the *attribute* "infinity" containing the number that is used to specify that one of the bounds (entries of $\underline{b}$, $\bar{b}$, l or u) is $\pm\infty$;
- the *dimension* "num_vars" containing the number n of Variable (columns in the matrix A) of the SimpleMILPBlock;
- the *dimension* "num_constraints" containing the number m of Constraint (rows in the matrix A) of the SimpleMILPBlock;
- the *dimension* "nzcount" containing the total number of nonzero coefficients in the Constraint of the SimpleMILPBlock;
- the *variable* "rmatval", of type double and indexed over the *dimension* "nzcount", that contains the nonzeros in the coefficient matrix A of the MILP, arranged row-wise (see rmatbeg);
- the *variable* "rmatind", of type UInt64 and indexed over the *dimension* "nzcount", that contains the column indices of the nonzeros in the coefficient matrix A of the MILP, arranged row-wise; that is, the i-th entry of rmatind indicates to which variable (column) the (allegedly, nonzero) coefficient to be found in the i-th entry of rmatval refers;
- the *variable* "rmatbeg", of type UInt64 and indexed over the *dimension* "num_constraints", that contains the indices into the rmatval/rmatind vectors where each constraint (row of the coefficient matrix A of the MILP) begins: all the nonzeros corresponding to constraint $h = 0, \ldots,$ **num_constraints** $- 1$ are to be found in rmatval[rmatbeg[h]], ..., rmatval[rmatbeg[$h+1$] -1], with their column (variable) indices to be found in the corresponding entries of rmatind[];
- the *variable* "x_lb", of type double and indexed over the *dimension* "num_vars", whose i-th entry is the (possibly, $-\infty$) lower bound on the feasible value of the i-th variable (the vector l);
- the *variable* "x_ub", of type double and indexed over the *dimension* "num_vars", whose i-th entry is the (possibly, $+\infty$) upper bound on the feasible value of the i-th variable (the vector u);
- the *variable* "row_lb", of type double and indexed over the *dimension* "num_constraints", whose i-th entry is the (possibly, $-\infty$) lower bound on the feasible value that the linear expression of the i-th constraint (as specified by the coefficients in rmatval, rmatind, and rmatbeg) can take (the vector $\underline{b}$);
- the *variable* "row_ub", of type double and indexed over the *dimension* "num_constraints", whose i-th entry is the (possibly, $+\infty$) upper bound on the feasible value that the linear expression of the i-th constraint (as specified by the coefficients in rmatval, rmatind, and rmatbeg) can take (the vector $\bar{b}$);
- the *variable* "obj", of type double and indexed over the *dimension* "num_vars", whose i-th entry is the coefficient of the i-th variable in the linear objective function of the MILP (the vector c);
- the scalar *variable* "obj_offset", of type double (and, since it is a scalar, not indexed over any dimension) giving the fixed offset in the linear objective function of the MILP (the constant $\bar{c}$).

All these are mandatory.

4.2 StructuredMILPBlock

While SimpleMILPBlock is a "leaf" Block just representing a generic MILP, StructuredMILPBlock derives from SimpleMILPBlock and extends it to represent "structured" MILP of the format

$$\begin{aligned} &\min \bar{c} + c^0 x^0 + \textstyle\sum_{h=1}^{k} c^k x^k \\ &\quad x^h \in X^h && h = 1, \ldots, k \\ &\quad \underline{b}^0 \leq A^0 x^0 \leq \bar{b}^0 \ , \ l^0 \leq x^0 \leq u^0 \\ &\quad \underline{r} \leq B^0 x^0 + \textstyle\sum_{h=1}^{k} B^k x^k \leq \bar{r} \end{aligned}$$

where X^h denotes the feasible region of the h-th sub-Block of the StructuredMILPBlock, which must be a SimpleMILPBlock, x^h its Variable, and c^h the coefficients of its objective. Thus, a StructuredMILPBlock has its own MILP constraints on its own variable x^0, represented exactly as in SimpleMILPBlock (indeed, a StructuredMILPBlock is a SimpleMILPBlock), plus the constraints of an arbitrary numbers of SimpleMILPBlock, plus constraints "linking" the variable x^0 of the "father" with those of the "sons". Note that the "sons" of a StructuredMILPBlock are SimpleMILPBlock, which means they can actually be other StructuredMILPBlock. Thus, StructuredMILPBlock allows to represent any block-structured MILP *provided that the linking constraints only link variables of the father to variables of its immediate sons, and not to variables of further descendants.*

Besides the mandatory "type" attribute of any :Block, and whatever has to be found in the netCDF::NcGroup of a SimpleMILPBlock (since a StructuredMILPBlock is a SimpleMILPBlock), the group should contain the following:

- the *dimension* "linking_num_constraints" containing the number of "linking" Constraint (rows of the matrices B^h) of the StructuredMILPBlock;
- the *dimension* "linking_nzcount" containing the total number of nonzero coefficients in the "linking" Constraint (the matrices B^h) of the StructuredMILPBlock;
- the *variable* "linking_rmatval", of type double and indexed over the *dimension* "linking_nzcount", that contains the nonzeros in the coefficient matrix of the "linking" Constraint (the matrices B^h), arranged row-wise (see linking_rmatbeg);
- the *variable* "linking_rmatind", of type UInt64 and indexed over the *dimension* "linking_nzcount", that contains the column indices of the nonzeros in the coefficient matrix of the "linking" Constraint (the matrices B^h), arranged row-wise; this has to be used together with linking_rmatblk, see below;
- the *variable* "linking_rmatblk", of type UInt64 and indexed over the *dimension* "linking_nzcount", that contains the block indices of the variables corresponding to nonzeros in the coefficient matrix of the "linking" Constraint (the matrices B^h); that is, the i-th entry of linking_rmatblk indicates the index h of the Block to which the Variable belongs, with $h = 0$ used for the "father" StructuredMILPBlock and $h = 1, 2, \ldots$ used for the Variable of the 1st, 2nd, ... of its SimpleMILPBlock sons, while the i-th entry of linking_rmatind is the index of the Variable in the h-th Block;
- the *variable* "linking_rmatbeg", of type UInt64 and indexed over the *dimension* "linking_num_constraints", that contains the indices into the linking_rmatval/linking_rmatind vectors where each "linking" Constraint (row of the matrices B^h) begins: all the nonzeros corresponding to constraint $h = 0, ...,$ **linking_num_constraints**-1 are to be found in linking_rmatval[linking_rmatbeg[h]], ..., linking_rmatval[linking_rmatbeg[$h+1$] -1], with their column (variable) and Block indices to be found in the corresponding entries of linking_rmatind and linking_rmatblk, respectively;
- the *variable* "linking_row_lb", of type double and indexed over the *dimension* "linking_num_constraints", whose i-th entry is the (possibly, $-\infty$) lower bound on the feasible value that the linear expression of the i-th "linking" Constraint (as specified by the coefficients in linking_rmatval, linking_rmatind, linking_rmatbeg, and linking_rmatblk) can take;
- the *variable* "linking_row_ub", of type double and indexed over the *dimension* "linking_num_constraints", whose i-th entry is the (possibly, $+\infty$) upper bound on the feasible value that the linear expression of the i-th

"linking" `Constraint` (as specified by the coefficients in `linking_rmatval`, `linking_rmatind`, `linking_rmatbeg`, and `linking_rmatblk`) can take.

All these are mandatory.

4.3 `MCFBlock`

The `MCFBlock` class implements the `Block` concept for the (linear) Min-Cost Flow (MCF) problem.

The data of the problem consist of a (directed) graph $G = (N, A)$ with $n = |N|$ nodes and $m = |A|$ (directed) arcs. Each node i has a deficit $b[i]$, i.e., the amount of flow that is produced/consumed by the node: source nodes (which produce flow) have negative deficits and sink nodes (which consume flow) have positive deficits. Each arc (i, j) has an upper capacity $U[i, j]$ and a linear cost coefficient $C[i, j]$. Flow variables $X[i, j]$ represents the amount of flow to be sent on arc (i, j). Parallel arcs, i.e., multiple copies of the same arc (i, j) are in general allowed; it is expected that they have different costs (for otherwise they can be merged into a unique arc), but this is not strictly enforced. Multiple copies of some arc (i, j) can be seen as "total" flow cost on that arc being a piecewise-linear convex function. The formulation of the problem is:

$$\begin{aligned} \min\ & \textstyle\sum_{(i,j)\in A} C[i,j]X[i,j] & \\ & \textstyle\sum_{(j,i)\in A} X[j,i] - \sum_{(i,j)\in A} X[i,j] = b[i] & \quad i \in N \\ & 0 \le X[i,j] \le U[i,j] & \quad (i,j) \in A \end{aligned}$$

The n equations are the flow conservation constraints and the $2m$ inequalities are the flow nonnegativity and capacity constraints. At least one of the flow conservation constraints is redundant, as the demands must be balanced ($\sum_{i\in N} b[i] = 0$); indeed, exactly n - ConnectedComponents(G) flow conservation constraints are redundant, as demands must be balanced in each connected component of G.

The graph G is allowed to be "partly dynamic". The set of nodes and arcs that are input at the beginning are assumed not to be changed (save for changing costs, capacities, and deficits, and for arcs to be closed or opened). Then, new arcs and nodes, up to a set maximum, can be dynamically added or deleted. This means that the graph can be fully static (if the maximum number of dynamic arcs and nodes is set to zero) as well as fully dynamic (if the initial graph is empty).

Besides the mandatory "`type`" attribute of any `:Block`, the *group* representing the `MCFBlock` should contain the following:

- the *dimension* "`NNodes`" containing the number of nodes in the graph (n);
- the *dimension* "`NArcs`" containing the number of arcs in the graph (m);
- the *variable* "`C`", of type `double` and indexed over the dimension "`NArcs`"; the i-th entry of the variable is assumed to contain the upper capacity of the i-th arc in the graph, that whose starting node and ending node are specified by the *variable* "`SN`" and "`EN`" (below);
- the *variable* "`U`", of type `double` and indexed over the dimension "`NArcs`"; the i-th entry of the variable is assumed to contain the cost of the i-th arc in the graph (the lower capacity being fixed to 0), that whose starting node and ending node are specified by the *variable* "`SN`" and "`EN`" (below);
- the *variable* "`B`", of type `double` and indexed over the dimension "`NNodes`"; the i-th entry of the variable is assumed to contain the deficit of the i-th node in the graph (note that node names here go from 0 to $n - 1$);
- the *variable* "`SN`", of type `int` and indexed over the dimension "`NArcs`"; the i-th entry of the variable is assumed to contain the starting node of the i-th arc in the graph (note that node names here go from 1 to n, they are shifted by 1 w.r.t. to the indices used in the "`B`" variable);
- the *variable* "`EN`", of type `int` and indexed over the dimension "`NArcs`"; the i-th entry of the variable is assumed to contain the ending node of the i-th arc in the graph (note that node names here go from 1 to n, they are shifted by 1 w.r.t. to the indices used in the "`B`" variable);

- the *dimension* "DynNNodes" containing the current number of dynamic nodes: all the nodes between 0 and "NNodes" − "DynNNodes" −1 are static, i.e., they cannot be deleted (and re-created), whereas all those from "NNodes" + "DynNNode" to "NNodes" −1 are dynamic, i.e., they can deleted and later on re-created;
- the *dimension* "DynNArcs" containing the current number of dynamic arcs: all the arcs between 0 and "NArcs" - "DynNArcs" −1 are static, i.e., they cannot be deleted (and re-created), whereas all those from "NArcs" + "DynNArcs" to "NArcs" −1 are dynamic, i.e., they can deleted and later on re-created;
- the *dimension* "MaxDynNNodes" containing the maximum number of dynamic nodes in the graph (the maximum value that n can take); data in the MCFBlock is allocated to accommodate for the fact that further "MaxDynNNodes" − "DynNNodes" nodes can later on be dynamically created (and deleted); if "MaxDynNNodes" < "DynNNodes" the dimension is ignored and "NNodes" is taken as the maximum overall number of nodes;
- the *dimension* "MaxDynNArcs" containing the maximum number of dynamic arcs in the graph (the maximum value that m can take): data in the MCFBlock is allocated to accommodate for the fact that further "MaxDynNArcs" − "DynNArcs" arcs can later on be dynamically created (and deleted); if "MaxDynNArcs" < "DynNArcs" the dimension is ignored and "NArcs" is taken as the maximum overall number of arcs.

The two *dimensions* "NNodes" and "NArcs" are mandatory, such as are the two *variables* "SN" and "EN". The three other *variables* are optional. If "C" is missing, all arc costs are assumed to be 0. If "U" is missing, all arc capacities are assumed to be infinite. If "B" is missing, all node deficits are assumed to be 0. Finally, all the *dimensions* "DynNNodes", "DynNArcs", "MaxDynNNodes" and "MaxDynNArcs" are optional: if they are missing they are treated as being 0 (this happening for all four means that the graph is "fully static" and cannot be changed).

4.4 UCBlock

The class UCBlock, implements the Block concept [see Block.h] for the Unit Commitment (UC) problem in electrical power production. This is typically a short-term (across for instance one week or one day time horizon) *deterministic* problem regarding finding an optimal schedule of the production of electrical generators satisfying a (large) set of technical constraints. The model is quite flexible due to the fact that different types of units and network constraints can be used by means of the fact that the class manages son Block of type UnitBlock and NetworkBlock. Also UCBlock handles a reasonably large variety of constraints, regarding not only active power but also primary and secondary reserve and inertia. Admittedly, some choices in UCBlock (like HeatBlock, pollution constraints, ...) are quite specific of the UC of the plan4res project; however, all the "nonstandard" aspects of UC can be switched away from the model (by simply not providing the data describing them). The main elements that UCBlock handles are:

- The time horizon of the problem, i.e., a discrete set of (typically, equally-spaced) time instants at which decisions are made (like, the 24 hours in a day);
- A set of electricity generating units, represented by derived classes of the base class UnitBlock;
- A set of NetworkBlock, one for each time instant in the time horizon, which represent the constraints on the electricity demand satisfaction and the technical constraints on the transmission network. These can be basically "empty" if the capacity of the transmission network is such as to never really impact generation decisions (a "bus");
- An optional set of HeatBlock, each representing the satisfaction of some specific "type of heat" on a close geographical area by heat-generating units possibly coupled with a heat storage. The link with the rest of the UC model lies in the fact that some of the heat-generating units in a HeatBlock may also be electricity generating ones (i.e., a UnitBlock); actually, the same UnitBlock can generate heat of "different types", and therefore appear as a heat-generating units in more than one HeatBlock.

Besides the mandatory cstrtype attribute of any :Block, the *group* should contain the following:

- the *dimension* "TimeHorizon" containing the number of time steps in the problem;
- the *dimension* "NumberUnits" containing the number of units (UnitBlock) in the problem;

- the *groups* "UnitBlock_0", "UnitBlock_1", ... , "UnitBlock_n" with n == NumberUnits - 1, containing each one UnitBlock corresponding to one or more electricity generating unit (electrical generator). It is an error if the corresponding groups are not there. Each UnitBlock can have more than one electrical generator (cf. UnitBlock::get_number_generators()), a value that is useful in the following (cf. "GeneratorNode") is the total number of those, which we will refer to as "NumberElectricalGenerators". This is computed by just calling get_number_generators() on each of the UnitBlock and summing all the results. Clearly, NumberElectricalGenerators >= NumberUnits and indeed, most of the UnitBlock can be expected to have just one electrical generator. If this happens for all the units then the NumberElectricalGenerators == NumberUnits. If, instead, some UnitBlock (like cascades of hydro generators or combined cycle plants) actually has more than one electrical generator, then NumberElectricalGenerators > NumberUnits (each UnitBlock must have at least one). This is because some UnitBlock (like cascades of hydro generators or combined cycle plants) can actually have more than one electrical generator in it (but each UnitBlock must have at least one). It is then useful (cf. "GeneratorNode") to be able to assign a unique index g = 0, 1, ..., NumberElectricalGenerators - 1 to each of the electrical generators in the UCBlock. When NumberElectricalGenerators == NumberUnits, the index is the same as i = 0, 1, ..., NumberUnits - 1(there is a one-to-one correspondence between UnitBlock and electrical generators). When, instead, NumberElectricalGenerators > NumberUnits, a mapping must be defined. The mapping is the obvious one: UnitBlock have an ordering i = 0, 1, ..., NumberUnits - 1 (cf. the groups "UnitBlock_0", "UnitBlock_1", ... above), and the electrical generators into each UnitBlock also have some natural ordering (corresponding to the columns of the matrices of variables, cf. e.g. UnitBlock::get_commitment()). Thus, in general the mapping is:

  - electrical generator 0 = first generator of UnitBlock_0
  - electrical generator 1 = second generator of UnitBlock_0
  - ...
  - electrical generator k = k-th generator of UnitBlock_0 (k = UnitBlock_0 -> get_number_generators()
  - electrical generator k + 1 = first generator of UnitBlock_1
  - electrical generator k + 2 = second generator of UnitBlock_1
  - ...

  which of course boils down to "g = i" when each UnitBlock has exactly one electrical generator;

- the *dimension* "NumberHeatBlocks" containing the number of heat blocks in the problem. The dimension is optional: if it is not provided then it is taken to be 0, which means that there is no heat block in the problem;

- the *groups* "HeatBlock_0", "HeatBlock_1", ... , "HeatBlock_n" with n == NumberHeatBlocks - 1, containing each one a HeatBlock. When NumberHeatBlocks == 0, these groups need not be there since they are not read. If, instead, NumberHeatBlocks > 0, it is an error if the corresponding groups are not there. Since each HeatBlock can have more than one heat generator (cf. HeatBlock::get_number_heat_generators()), a value that is useful in the following (cf. "HeatSet") is the total number of those. We will refer to such number as "NumberHeatGenerators", which is computed by just calling get_number_heat_generators() on each of the HeatBlock and summing all the results. Clearly, NumberHeatGenerators >= NumberHeatBlock. Some of the HeatBlock may have just one heat generator; if this happens for all the heat blocks (but this is not likely), then NumberHeatGenerators == NumberHeatBlocks. It is then useful (cf. "HeatNode") to be able to assign a unique index h = 0, 1, ..., NumberHeatGenerators - 1 to each of the heat generators in the UCBlock. When NumberHeatGenerators == NumberHeatBlocks the index is the same as b = 0, 1, ..., NumberHeatBlock - 1 (there is a one-to-one correspondence between HeatBlock and heat generators, but this is not likely to happen). When, instead, NumberHeatGenerators > HeatBlock, a mapping must be defined. The mapping is the obvious one: HeatBlock have an ordering b = 0, 1, ..., NumberHeatBlocks - 1 (cf. the groups "HeatBlock_0", "HeatBlock_1", ... above), and the heat generators into each HeatBlock also have some natural ordering (corresponding to the columns of the matrices of variables, cf. e.g. HeatBlock::get_heat(). Thus, in general the mapping is:

  - heat generator 0 = first generator of HeatBlock_0
  - heat generator 1 = second generator of HeatBlock_0

  - ...
  - heat generator k = k-th generator of HeatBlock_0 (k = HeatBlock_0 -> get_number_heat_units()
  - heat generator k + 1 = first generator of HeatBlock_1
  - heat generator k + 2 = second generator of HeatBlock_1
  - ...

  which of course boils down to "h = b" when each HeatBlock has exactly one heat generator (but this is not assumed to happen);

- optionally, the dimensions and variables necessary to deserialize a NetworkData object that describes the transmission network; see NetworkData::NetworkData::deserialize() for details. If that is not provided (basically, "NumberNodes" is not provided or it is == 1), then the transmission network is taken to have only one node (a bus);

- the *groups* "NetworkBlock_0", "NetworkBlock_1", ... , "NetworkBlock_t" with t = TimeHorizon - 1, containing each the constraints on the transmission network at time t;

- the *variable* "GeneratorNode", of type int and indexed over the set {0, ... , "NumberElectricalGenerators" - 1}; GeneratorNode[g] tells to which node of the transmission network, the specified electrical generator g belongs. Note that this means that different electrical generators in the same UnitBlock can belong to different nodes of the transmission network. This is justified e.g. by hydro cascade units where different turbines can be rather far apart geographically, but still linked by (long) stretches of rivers. If NumberElectricalGenerators == NumberUnits (all UnitBlock have exactly one electrical generator), then this variable is indexed over NumberUnits. If NumberNodes == 1 (say, it is not provided at all), then this variable need not be defined, since it is not loaded.

- the *variable* "HeatNode", of type int and indexed over the *dimension* "NumberHeatBlocks"; the entry HeatNode[h] tells to which node of the transmission network *all* the heat generators that are also electrical generators in the HeatBlock h belong. Note that this implies that, unlike electrical generators in a UnitBlock, heat generators in a HeatBlock are always "geographically near to each other" so that they are necessarily attached to the same node of the transmission network (which is a reasonable assumption since heat, unlike say water, usually cannot travel much far). If NumberHeatBlocks == 0 (say, it is not provided at all), then this variable need not be defined, since it is not loaded. This information is actually only used to determine in which Pollutant Zone a HeatBlock is located, in order to add the corresponding heat generators (that are not also electricity generators) to the pollution constraints. This means that also if NumberPollutants == (say, it is not provided at all) this variable is useless and therefore need not be defined, since it is not loaded. Finally, notice that for heat generators into a HeatBlock that also are electrical generators, this variable provides again an information that is already known, i.e., to which node they belong to (cf. "GeneratorNode" above). Of course *the two information must agree*, otherwise the input file is ill-defined and exception is thrown.

- the *variable* "HeatSet", of type int and indexed over the set {0, ..., "NumberHeatGenerators" - 1}. If HeatSet[h] = k < NumberElectricalGenerators, then k is the unique name of the electrical generator corresponding to the heat generator h; otherwise the heat generator h is not also an electrical generator. If NumberHeatBlocks == 0 (there is no HeatBlock) then this variable need not be defined, since it is not loaded. It is possible that different heat generators correspond to the same electrical generator (that is, HeatSet[h1] == HeatSet[h2] for some h1 != h2), but not vice-versa: a heat generator either corresponds to a specific electrical generator, or to no electrical generator (it is an heat-only generator);

- the *variable* "PowerHeatRho", of type double and indexed over the *dimension* "NumberUnits": entry PowerHeatRho[i] is assumed to contain the electrical-power-to-heat ratio for *all generators* within unit i (most likely, a single generator). This is only used in the constraints linking electrical units to heat units, hence if NumberHeatBlocks == 0 (there are no HeatBlock) then this variable need not be defined, since it is not loaded;

- the *dimension* "NumberPrimaryZones" tells how many "primary spinning reserve zones" are there in the problem. The *dimension* is optional, if it is not provided then it is taken to be 0, which means that no primary reserve constraints are present in the problem;

- the *variable* "PrimaryZones", of type int and indexed over the *dimension* "NumberNodes". The entry PrimaryZones[i] tells to which primary zone the node i belongs: if PrimaryZones[i] >= NumberPrimaryZo this means that node i does not belong to any primary zone, and hence the corresponding electrical generators are not involved into the primary reserve constraints. If NumberPrimaryZones == 0 (say, it is not provided at all) then this variable need not be defined, since it is not loaded. If NumberPrimaryZones == 1 and this variable is not defined, then there is only one primary zone and all the nodes belong to it;

- the *variable* "PrimaryDemand", of type double and indexed both over the *dimensions* "NumberPrimaryZones" and "TimeHorizon": entry PrimaryDemand[i , t] is assumed to contain the primary reserves requirement which are specified on the primary reserve zone i in the time t. If NumberPrimaryZones == 0 (say, it is not provided at all), then this variable need not be defined, since it is not loaded;

- the *dimension* "NumberSecondaryZones" tells how many "secondary spinning reserve zones" are there in the problem. The *dimension* is optional, if it is not provided then it is taken to be 0, which means that no secondary reserve constraints are present in the problem;

- the *variable* "SecondaryZones", of type int and indexed over the *dimension* "NumberNodes". The entry SecondaryZones[i] tells to which secondary zone the node i belongs: if SecondaryZones[i] >= NumberSecondaryZones, this means that node i does not belong to any secondary zone, and hence the corresponding electrical generators are not involved into the secondary reserve constraints. If NumberSecondaryZones == 0 (say, it is not provided at all) then this variable need not be defined, since it is not loaded. If NumberSecondaryZone == 1 and this variable is not defined, then there is only one secondary zone and all the nodes belong to it;

- the *variable* "SecondaryDemand", of type double and indexed both over the *dimensions* "NumberSecondaryZones" and "TimeHorizon": entry SecondaryDemand[i , t] is assumed to contain the secondary reserves requirement which are specified on the secondary reserve zone i in the time t. If NumberSecondaryZones == 0 (say, it is not provided at all), then this variable need not be defined, since it is not loaded;

- the *dimension* "NumberInertiaZones" tells how many "inertia constraints zones" are there in the problem. The *dimension* is optional, if it is not provided then it is taken to be 0, which means that no inertia constraints are present in the problem;

- the *variable* "InertiaZones", of type int and indexed over the *dimension* "NumberNodes". The entry InertiaZones[n] tells to which inertia zone the node n belongs: if InertiaZones[n] >= NumberInertiaZor this means that node n does not belong to any inertia zone, and hence the corresponding units are not involved into the inertia reserve constraints. If NumberInertiaZones == 0 (say, it is not provided at all) then this variable need not be defined, since it is not loaded. If NumberInertiaZones == 1 and this variable is not defined, then there is only one inertia zone and all the nodes belong to it;

- the *variable* "InertiaDemand", of type double and indexed both over the *dimensions* "NumberInertiaZones" and "TimeHorizon": entry InertiaDemand[i , t] is assumed to contain the inertia reserves requirement which are specified on the inertia reserve zone i in the time t. If NumberInertiaZones == 0 (say, it is not provided at all), then this variable need not be defined, since it is not loaded;

- the *dimension* "NumberPollutants" containing the number of pollutants in the problem. The *dimension* is optional, if it is not provided then it is taken to be 0, which means that no pollutants constraints are present in the problem;

- the *variable* "NumberPollutantZones" of type int and indexed over the *dimension* "NumberPollutants": the entry NumberPollutantZones[p] is assumed to contain the number of pollutant zones associated with pollutant p. If NumberPollutants == 0 (say, it is not provided) then this variable need not be defined, since it is not loaded. Then, all number of pollutant zones associated with each pollutant p is computed as: TotalNumberPollutantZones == NumberPollutantZone[0] + ...+ NumberPollutantZone[Number - 1];

- the *variable* "PollutantZones", of type int and indexed over the *dimensions* "NumberPollutants" and "NumberNodes": the entry PollutantZones[p , n] tells to which pollutant zone associated with pollutant p the node n belongs. If PollutantZones[p , n] >= NumberPollutantZones[p], this means that node n does not belong to any pollutant zone, and hence the corresponding units are not involved

into the pollutant budget constraints associated with pollutant p. If `NumberPollutants == 0` (say, it is not provided) then this variable need not be defined, since it is not loaded;

- the *variable* "`PollutantBudget`", of type double and indexed over the set { 0 , ..., `TotalNumberPollutantZones - 1` }: the entry `PollutantBudget[ n ]` for `n == 0 , ..., TotalNumberPollutantZones - 1` is assumed to contain the pollutant budget (across all the time horizon) for the pair (zone of the pollutant , pollutant) corresponding to n. In another word, since the number of pollutant zones of each pollutant may not be equal with each otherit is useful (to avoid having to store PollutantBudget as an irregular matrix) to be able to assign a unique index `n = 0, 1, ..., TotalNumberPollutantZones - 1` to each pollutant budget of each pollutant zone. A mapping must be defined between each entry n and the pair (zone of the pollutant , pollutant). The mapping is the obvious one: `UCBlock` has a set of pollutants `p = 0, 1, ..., NumberPollutants - 1`, and each pollutant p may have several pollutant zones (see comments of variable "`NumberPollutantZones`" above). Thus, in general the mapping is:

  - `n = 0` corresponds to the zone 0 of pollutant 0
  - `n = 1` corresponds to the zone 1 of pollutant 0
  - `...`
  - `n = NumberPollutantZone[ 0 ] - 1` corresponds to the zone `NumberPollutantZone[ 0 ] - 1` of pollutant 0
  - `n = NumberPollutantZone[ 0 ]` corresponds to the zone 0 of pollutant 1
  - `n = NumberPollutantZone[ 0 ] + 1` corresponds to the zone 1 of pollutant 1
  - `...`

  If `NumberPollutants == 0` (say, it is not provided) then this variable need not be defined, since it is not loaded;

- the *variable* "`PollutantRho`", of type double and indexed over two *dimensions* which are "`TimeHorizon`" and "`NumberPollutants`" and the set {0,..., `NumberElectricalGenerators - 1`} (see comments above). The first dimension can have size either 1 or "`TimeHorizon`". In the former case the entry `PollutantRho[ 0 , p , g ]` is assumed to contain the conversion factor of pollutant p due to the electrical generator g which is equal for all time instants t. Otherwise, the first dimension has full size `TimeHorizon` and the entry `PollutantRho[ t , p , g ]` gives the conversion factor of pollutant p due to the electrical generator g for time t. If `NumberPollutants == 0` (it is not provided) then this variable need not be defined, since it's not loaded.

- the *variable* "`PollutantHeatRho`", of type double and indexed over three *dimensions*. The first *dimension* can have size 1 or "`TimeHorizon`". The second and third *dimensions* have sizes "`NumberPollutants`" and "`NumberHeatBlocks`", respectively. If the first *dimension* has size 1, then the entry `PollutantRho[ 0 , p , h ]` is assumed to contain the conversion factor of pollutant p due to the generation of *all* heat generators in `HeatBlock` h, which is equal for all time instants t. If, instead, the first *dimension* has full size "`TimeHorizon`", then the entry `PollutantRho[ t , p , h ]` gives the conversion factor of pollutant p due to the generation of *all* heat generators in `HeatBlock` h for time instant t. If `NumberPollutants == 0` (say, it is not provided) then this variable does not need be defined, since it is not loaded. If `NumberHeatBlocks == 0` (there is no `HeatBlock`) then this variable need not be defined, since it is not loaded.

4.5 `UnitBlock`

The `UnitBlock` class, which derives from the `Block`, defines a base class for any possible "`unit`" that can be attached to a `UCBlock`. A unit is in general a set of electrical generators tied together by some technical constraints, although many units actually correspond to only one generator. The base `UnitBlock` class only has very basic information that can characterize almost any different kind of unit:

- The time horizon of the problem;
- The number of generators in the unit (1 by default, see `get_number_generators()`);
- Four `boost::multi_array< ColVariable, 2 >` objects, that are used to store the information regarding:

- the commitment of the generators in the unit;
- the primary spinning reserve of the generators in the unit;
- the secondary spinning reserve of the generators in the unit;
- the active power produced by the generators in the unit;

Each of the `boost::multi_array< ColVariable, 2 >` has as first *dimension* the time horizon and as second *dimension* the number of generators (as returned `get_number_generators()`). There are two possible cases:

- if some `boost::multi_array< ColVariable, 2 >` is empty(), then the corresponding variable does not exist (for instance, the generator in the unit may not have reserve);
- otherwise, the `boost::multi_array< ColVariable, 2 >`, say M, must have `f_time_horizon` rows and `get_number_generators()` columns: each element $M[t, g]$ gives the variable for time step t of generator g.

The class also outputs some general information regarding how the active power and/or commitment status of each generator of the unit at a given time instant impact the unit's capability of satisfying inertia constraints, and the fixed consumption (if any) of each generator in the unit when it is off. Besides the mandatory "`type`" attribute of any `:Block`, the *group* representing the `UnitBlock` should contain the following:

- the *dimension* "`TimeHorizon`" containing the time horizon. The *dimension* is optional because the same information may be passed via the method `set_time_horizon()`, or directly retrieved from the father if it is a `UCBlock`; see the comments to `set_time_horizon()` for details;
- the *dimension* "`NumberIntervals`" that is provided to allow that all time-dependent data in the `UnitBlock` can only change at a subset of the time instants of the time interval, being therefore piecewise-constant (possibly, constant). "`NumberIntervals`" should therefore be <= "`TimeHorizon`", with three distinct cases:
  - "`NumberIntervals`" <= `1`, which is taken to mean "`NumberIntervals`" == `1`; this is what is assumed if the *dimension*, that is optional, is not there. This means that the value of each relevant data in the `UCBlock` is the same for each time instant `0, ...,` "`TimeHorizon`" `- 1` in the time horizon. In this case, the variable "`ChangeIntervals`" (see below) is ignored;
  - `1 <` "`NumberIntervals`" < "`TimeHorizon`", which means that in some time instants, *but not all of them*, the values of some of the relevant data are changing; the intervals are then described in variable "`NumberIntervals`";
  - "`NumberIntervals`" == "`TimeHorizon`", which means that values of the relevant data changes at every time interval (in principle; of course there is nothing preventing the same value to be repeated in the `netCDF` input). Also in this case the variable "`NumberIntervals`" is ignored, since it is useless;

  Note that this (together with "`NumberIntervals`", if defined) obviously sets the "`maximum frequency`" at which data can change; if some data changes less frequently (say, it is constant), then the same value will have to be repeated. Individual data can also have specific provisions for the case where the data is all equal despite "`NumberIntervals`" saying differently.
- the *variable* "`ChangeIntervals`", of type integer and indexed over the *dimension* "`NumberIntervals`". The time horizon is subdivided into `NumberIntervals = k` of the form $[0, i_1]$,$[i_1+1, i_2]$, $\dots$ $[i_{k-1}+1, TimeHorizon-1]$; "`ChangeIntervals`" then has to contain $[i_1, i_2, \dots i_{k-1}]$. Note that, therefore, "`ChangeIntervals`" has one significant value less than "`NumberIntervals`", which means that `ChangeIntervals[ NumberIntervals - 1 ]` is ignored. Anyway, the whole variable is ignored if either "`NumberIntervals`" <= `1` (such as if it is not defined), or "`NumberIntervals`" >= "`TimeHorizon`".

4.5.1 `ThermalUnitBlock`

The `ThermalUnitBlock` class derives from `UnitBlock` and implements a "`reasonably standard`" thermal unit of a Unit Commitment Problem. That is, the class is designed in order to give mathematical formulation to describe the operation of large set of conventional power plants (such as nuclear, hard coal, gas turbine, gas, combined cycle, oil, ...) which are directly connected to the transmission grid. Besides the mandatory "`type`" attribute of any `:Block`, the group must contain all the data required by the base `UnitBlock`, as described in the comments

to UnitBlock::deserialize(netCDF::NcGroup). In particular, we refer to that description for the crucial *dimensions* "TimeHorizon", "NumberIntervals" and "ChangeIntervals". The netCDF::NcGroup must then also contain:

- the *variable* "MinPower", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector MnP[t] that, for each time instant t, contains the minimum active power output value of the unit for the corresponding time step. If "MinPower" has length 1 then MnP[t] contains the same value for all t. Otherwise, MinPower[i] is the fixed value of MnP[t] for all t in the interval [ChangeIntervals[i - 1] , ChangeIntervals[i]], with the assumption that ChangeIntervals[- 1] = 0. Note that it must be MnP[t] >= 0 for all t. If NumberIntervals <= 1 or NumberIntervals >= TimeHorizon, then the mapping clearly does not require "changeIntervals", which in fact is not loaded;

- the *variable* "MaxPower", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector MxP[t] that, for each time instant t, contains the maximum active power output value of the unit for the corresponding time step. If "MaxPower" has length 1 then MxP[t] contains the same value for all t. Otherwise, MaxPower[i] is the fixed value of MxP[t] for all t in the interval [ChangeIntervals[i - 1] , ChangeIntervals[i]], with the assumption that ChangeIntervals[- 1] = 0. Note that it must be MxP[t] >= 0 for all t. If NumberIntervals <= 1 or NumberIntervals >= TimeHorizon, then the mapping clearly does not require "changeIntervals", which in fact is not loaded;

- the *variable* "DeltaRampUp", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector DP[t] that, for each time instant t, contains the ramp-up value of the unit for the corresponding time step, i.e., the maximum possible increase of active power production w.r.t. the power that had been produced in time instant t - 1, if any. This variable is optional; if it is not provided then it is assumed that DP[t] == MxP[t], i.e., the unit can ramp up by an arbitrary amount, i.e., there are no ramp-up constraints. If "DeltaRampUp" has length 1 then DP[t] contains the same value for all t. Otherwise, DeltaRampUp[i] is the fixed value of DP[t] for all t in the interval [ChangeIntervals[i - 1] , ChangeIntervals[i]], with the assumption that ChangeIntervals[- 1] = 0. If NumberIntervals <= 1 or NumberIntervals >= TimeHorizon, then the mapping clearly does not require "ChangeIntervals", which in fact is not loaded.

- the *variable* "DeltaRampDown", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector DM[t] that, for each time instant t, contains the ramp-down value of the unit for the corresponding time step, i.e., the maximum possible decrease of active power production w.r.t. the power that had been produced in time instant t - 1, if any. This variable is optional; if it is not provided then it is assumed that DM[t] == MxP[t], i.e., the unit can ramp down by an arbitrary amount, i.e., there are no ramp-down constraints. If "DeltaRampDown" has length 1 then DM[t] contains the same value for all t. Otherwise, DeltaRampDown[i] is the fixed value of DM[t] for all t in the interval [ChangeIntervals[i - 1] , ChangeIntervals[i]], with the assumption that ChangeIntervals[- 1] = 0. If NumberIntervals <= 1 or NumberIntervals >= TimeHorizon, then the mapping clearly does not require "ChangeIntervals", which in fact is not loaded.

- the *variable* "PrimaryRho", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector PR[t] that, for each time instant t, contains the maximum possible fraction of active power that can be used as primary reserve value of the unit for the corresponding time step. This variable is optional; if it is not provided then it is assumed that this unit may not be capable of producing any primary reserve, which correspond to PR[t] == 0 for all t. If "PrimaryRho" has length 1 then PR[t] contains the same value for all t. Otherwise, PrimaryRho[i] is the fixed value of PR[t] for all t in the interval [ChangeIntervals[i - 1] , ChangeIntervals[i]], with the assumption that ChangeIntervals[- 1] = 0. If NumberIntervals <= 1 or NumberIntervals >= TimeHorizon, then the mapping clearly does not require "ChangeIntervals", which in fact is not loaded.

- the *variable* "SecondaryRho", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector SR[t] that, for each time instant t, contains the maximum possible fraction of active power that can be used as secondary reserve value of the unit for the corresponding time step.

This variable is optional; if it is not provided then it is assumed that this unit may not be capable of producing any secondary reserve, which correspond to `SR[ t ] == 0` for all t. If "`SecondaryRho`" has length 1 then `SR[ t ]` contains the same value for all t. Otherwise, `SecondaryRho[ i ]` is the fixed value of `SR[ t ]` for all t in the interval `[ ChangeIntervals[ i - 1 ] , ChangeIntervals[ i ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`. If `NumberIntervals <= 1` or `NumberIntervals >= TimeHorizon`, then the mapping clearly does not require "`ChangeIntervals`", which in fact is not loaded.

- the *variable* "`QuadTerm`", of type double and either of size 1 or indexed over the *dimension* "`NumberIntervals`". This is meant to represent the vector `A[ t ]` that, for each time instant t, contains the quadratic term of power cost function of the unit for the corresponding time step. This variable is optional; if it is not provided then it is assumed that `A[ t ] == 0`, i.e., the cost of the unit is linear in the produced power. If "`QuadTerm`" has length 1 then `A[ t ]` contains the same value for all t. Otherwise, `QuadTerm[ i ]` is the fixed value of `A[ t ]` for all t in the interval `[ ChangeIntervals[ i - 1 ] , ChangeIntervals[ i ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`. If `NumberIntervals <= 1` or `NumberIntervals >= TimeHorizon`, then the mapping clearly does not require "`ChangeIntervals`", which in fact is not loaded.

- the *variable* "`StartUpCost`", of type double and either of size 1 or indexed over the *dimension* "`NumberIntervals`". This is meant to represent the vector `SC[ t ]` that, for each time instant t, contains the start up cost value of the unit for the corresponding time step. This variable is optional; if it is not provided then it is assumed that `SC[ t ] == 0`, i.e., this unit may not have any start up cost. If "`StartUpCost`" has length 1 then `SC[ t ]` contains the same value for all t. Otherwise, `StartUpCost[ i ]` is the fixed value of `SC[ t ]` for all t in the interval `[ ChangeIntervals[ i - 1 ] , ChangeIntervals[ i ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`. If `NumberIntervals <= 1` or `NumberIntervals >= TimeHorizon`, then the mapping clearly does not require "`ChangeIntervals`", which in fact is not loaded.

- the *variable* "`LinearTerm`", of type double and either of size 1 or indexed over the *dimension* "`NumberIntervals`". This is meant to represent the vector `B[ t ]` that, for each time instant t, contains the linear term of power cost function of the unit for the corresponding time step. This variable is optional; if it is not provided then it is assumed that `B[ t ] == 0`, i.e., the cost of the unit has no linear dependence on the produced power (say, only the quadratic one). If "`LinearTerm`" has length 1 then `B[ t ]` contains the same value for all t. Otherwise, `LinearTerm[ i ]` is the fixed value of `B[ t ]` for all t in the interval `[ ChangeIntervals[ i - 1 ] , ChangeIntervals[ i ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`. If `NumberIntervals <= 1` or `NumberIntervals >= TimeHorizon`, then the mapping clearly does not require "`ChangeIntervals`", which in fact is not loaded.

- the *variable* "`ConstTerm`", of type double and either of size 1 or indexed over the *dimension* "`NumberIntervals`". This is meant to represent the vector `C[ t ]` that, for each time instant t, contains the const term of power cost function of the unit for the corresponding time step. This variable is optional; if it is not provided then it is assumed that `C[ t ] == 0`, i.e., the cost of the unit has no fixed term, only those depending (linearly or quadratically) on the produced power. If "`ConstTerm`" has length 1 then `C[ t ]` contains the same value for all t. Otherwise, `ConstTerm[ i ]` is the fixed value of `C[ t ]` for all t in the interval `[ ChangeIntervals[ i - 1 ] , ChangeIntervals[ i ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`. If `NumberIntervals <= 1` or `NumberIntervals >= TimeHorizon`, then the mapping clearly does not require "`ChangeIntervals`", which in fact is not loaded.

- the scalar *variable* "`InitialPower`", of type double and not indexed over any *dimension*. This variable indicates the amount of the power that the unit was producing at time instant -1, i.e., before the start of the time horizon; this is necessary to compute the ramp-up and ramp-down constraints. Clearly, it must be that `MaxPower >= InitialPower >= MinPower` if the unit was "on" at time instant -1, and it must be that `InitialPower == 0` if the unit was "off" at time instant -1. The on/off status of the unit is also encoded by the scalar variable `InitUpDownTime`: in particular, `InitUpDownTime > 0` then the unit was on at time instant -1, and therefore `InitialPower >= MinPower` must hold, while if `InitUpDownTime <= 0` then the unit was off at time instant -1, and therefore `InitialPower == 0` by definition. In fact, if `InitUpDownTime <= 0` then this variable need not be defined since it is not loaded.

- the scalar *variable* "`InitUpDownTime`", of type Int64 and not indexed over any *dimension* and indicates the initial time to generating the unit. If `InitUpDownTime > 0`, this means that the unit has been on for `InitUpDownTime` time stamps prior to time stamp 0 (the beginning of the horizon). If, instead, `InitUpDownTime`

<= 0, this means that the unit has been off for - InitUpDownTime time stamps prior to time stamp 0; note that InitUpDownTime == 0 means that the unit has been just shut down at the end of time instant −1, i.e., the beginning of time instant 0;

- the positive scalar *variable* "MinUpTime", of type UInt64 and not indexed over any *dimension*, which indicates the minimum allowed up time in this unit. This variable is optional, if it is not provided it is taken to be MinUpTime == 0, which mean that the unit can shut down in the very same time stamp in which it starts up.
- the positive scalar *variable* "MinDownTime", of type UInt64 and not indexed over any *dimension*, which indicates the minimum allowed down time in this unit. This variable is optional, if it is not provided it is taken to be MinDownTime == 0, which mean that the unit can start up in the very same time stamp in which it starts up.
- the *variable* "FixedConsumption", of type double and either of size 1 or indexed over the *dimension* "NumberInterval
  This is meant to represent the vector FC[t] that, for each time instant t, contains the fixed consumption of the power plant if it is OFF at time t. The variable is optional; if it is not defined, FC[t] == 0 for all time instants. If it has size 1, then FC[t] == FixedConsumption[0] for all t, regardless to what "NumberIntervals" says. Otherwise, FixedConsumption[i] is the fixed value of FC[t] for all t in the interval [ChangeIntervals[i - 1], ChangeIntervals[i]], with the assumption that ChangeIntervals[-1] = 0.
- the *variable* "InertiaCommitment", of type double and either of size 1 or indexed over the *dimension* "NumberIntervals". This is meant to represent the vector IC[t] that, for each time instant t, contains the contribution that the unit can give to the inertia constraint for the sole fact that is is on (basically, the constant to be multiplied to the commitment variable) at time t. The variable is optional; if it is not defined, IC[t] == 0 for all time instants. If it has size 1, then IC[t] == InertiaCommitment[0] for all t, regardless to what "NumberIntervals" says. Otherwise, InertiaCommitment[i] is the fixed value of IC[t] for all t in the interval [ChangeIntervals[i - 1], ChangeIntervals[i]], with the assumption that ChangeIntervals[-1] = 0.

4.5.2 HydroUnitBlock

4.6 NetworkBlock

The class NetworkBlock, which derives from the Block, defines the basic interface for the constraints/optimization problems which describe the behaviour of the transmission network in a specific time instant in the Unit Commitment (UC) problem, as represented in UCBlock. The base class handles only basic information: it allows to read/set the topology (and capacity/susceptances) of the network, and the active power demand at the different nodes in the given time instant. This information is actually bunched together in a small "passive" NetworkData object (no methods, just a data repository) that can be either de-serialized or passed ready-made (typically, by the UCBlock). Details of the kind of network that is implemented ("bus", DC equations, AC equations, OPF, ...) are entirely demanded to derived objects. The interface between a NetworkBlock and the rest of the UC is just the vector of node injection variables, which will have to satisfy the technical constraints of the transmission network. The NetworkData class is a nested sub-class which only serves to have a quick way to load all the basic data (topology and electrical characteristics) that describe the transmission network. The rationale is that while often the network does not change during the (short) time horizon of UC, it makes sense to allow for this to happen. This means that individual NetworkBlock objects may in principle have different NetworkData, but most often they can share the same. By bunching all the information together we make it easy for this sharing to happen. Deserialize a NetworkData out of a netCDF::NcGroup, which should contain the following:

- the *dimension* "NumberNodes" containing the number of nodes in the problem; this dimension is optional, if it is not provided then it is taken to be == 1; If NumberNodes == 1 (equivalently, it is not provided), the network is a "bus" formed of only one node, and therefore all the subsequent information need not to be present since it is not loaded. If NumberNodes > 1, then all the subsequent information is mandatory:
- the *dimension* "NumberLines" containing the number of lines in the transmission network;
- the *variable* "StartLine", of type int and indexed over the *dimension* "NumberNodes"; the $i-th$ entry of the variable is the starting point of the line (a number in 0, ..., NumberNodes - 1). Note that lines are

not oriented, but the flow of energy is; that is, a positive flow along line i means that energy is being taken away from `StartLine[ i ]` and delivered to `EndLine[ i ]` (see next), a negative flow means vice-versa. Note that node names here go from 0 to `NNodes.getSize() - 1`;

- the *variable* "`EndLine`", of type int and indexed over the *dimension* "`NumberNodes`"; the $i-th$ entry of the variable is the ending point of the line (a number in `0, ..., NumberNodes - 1`; lines are not oriented, but see above). `StartLine[ i ] == EndLine[ i ]` (a self-loop) is not allowed, but multiple lines between the same pair of nodes are. Note that node names here go from 0 to `NNodes.getSize() - 1`;
- the *variable* "`MinPowerFlow`", of type double and indexed over the *dimension* "`NumberLines`"; the $i-th$ entry of the variable is assumed to contain the minimum power flow on line i (note that this is typically a negative number as lines are bi-directional, see above);
- the *variable* "`MaxPowerFlow`", of type double and indexed over the *dimension* "`NumberLines`"; the $i-th$ entry of the variable is assumed to contain the maximum power flow at line i (a non-negative number);
- the *variable* "`Susceptance`", of type double and indexed over the *dimension* "`NumberLines`"; the $i-th$ entry of this variable is assumed to contain the susceptance of line i. Note that this is strictly a positive value.

4.6.1 `BusNetworkBlock`

The `BusNetworkBlock` class, which derives from `NetworkBlock` [see NetworkBlock.h] implements the `Block` concept [see Block.h] in order to define a "`bus`" transmission network in the Unit Commitment problem for a given instant in the time horizon. There is actually precious little that this class has to do that is not done already by the base `NetworkBlock` class.

4.6.2 `DCNetworkBlock`

The `DCNetworkBlock` class derives from `NetworkBlock`, and defines the standard linear constraints corresponding to the "`DC model`" of the transmission network in the Unit Commitment problem.

4.7 `HeatBlock`

The `HeatBlock` class implements the `Block` concept [see Block.h] for a set of "`heat units`" as required in the plan4res project. The constraints regarding heat management in the plan4res project can be described separately for each `HeatBlock`. In the parlance of plan4res, a HB is the intersection of an Energy Cell and a heat-ID: a set of heat-producing units that are geographically close enough to exchange heat of the same type, together with possibly a (single) storage for this type of heat, in order to satisfy a given heat demand. HBs are only connected to each other because some heat-producing units are also electricity-producing ones, where heat is a by-product or a co-product of electricity. However, this linking "`happens at the level of the main UC model`", and therefore it is not required for the description of the internal constraints of a HB. All this on a given time horizon, as in the UC problem. The operations of the heat generating unit are described on a discrete time horizon where there may exist some heat-producing units, possibly of a single heat storage, and of the demand that has to be satisfied. Besides the mandatory "`type`" attribute of any `:Block`, the group should contain all the following:

- the *dimension* "`NumberHeatUnits`" containing the number heat-producing units. This is not optional, and has to be `>= 1`;
- the *dimension* "`TimeHorizon`" containing the time horizon. The *dimension* is optional because the same information may be passed via the method `set_time_horizon()`, or directly retrieved from the father if it is a `UCBlock`; see the comments to `set_time_horizon()` for details;
- the *dimension* "`NumberIntervals`" that is provided to allow that all time-dependent data in the `HeatBlock` can only change at a subset of the time instants of the time interval, being therefore piecewise-constant (possibly, constant). "`NumberIntervals`" should therefore be `<=` "`TimeHorizon`", with three distinct cases:
  - "`NumberIntervals`" `<= 1`, which is taken to mean "`NumberIntervals`" `== 1`; this is what is assumed if the *dimension*, that is optional, is not there. This means that the value of each relevant data in the `UCBlock` (see e.g. "`CostHeatUnit`", "`MinHeatProduction`" and "`MaxHeatProduction`" below)is

the same for each time instant `0, ...,` "`TimeHorizon`" `- 1` in the time horizon. In this case, the variable "`ChangeIntervals`" (see below) is ignored;

- `1 <` "`NumberIntervals`" `<` "`TimeHorizon`", which means that in some time instants, *but not all of them*, the values of some of the relevant data are changing; the intervals are then described in variable "`NumberIntervals`";
- "`NumberIntervals`" == "`TimeHorizon`", which means that values of the relevant data changes at every time interval (in principle; of course there is nothing preventing the same value to be repeated in the `netCDF` input). Also in this case the variable "`NumberIntervals`" is ignored, since it is useless;

Note that this (together with "`NumberIntervals`", if defined) obviously sets the "`maximum frequency`" at which data can change; if some data changes less frequently (say, it is constant), then the same value will have to be repeated. Individual data can also have specific provisions for the case where the data is all equal despite "`NumberIntervals`" saying differently.

- the *variable* "`ChangeIntervals`", of type integer and indexed over the *dimension* "`NumberIntervals`". The time horizon is subdivided into `NumberIntervals = k` of the form $[0, i_1]$,$[i_1+1, i_2]$, $\dots$ $[i_{k-1}+1, TimeHorizon-1]$; "`ChangeIntervals`" then has to contain $[i_1, i_2, \dots i_{k-1}]$. Note that, therefore, "`ChangeIntervals`" has one significant value less than "`NumberIntervals`", which means that `ChangeIntervals[ NumberIntervals - 1 ]` is ignored. Anyway, the whole variable is ignored if either "`NumberIntervals`" `<= 1` (such as if it is not defined), or "`NumberIntervals`" `>=` "`TimeHorizon`";

- the *variable* "`TotalHeatDemand`", of type double and indexed over the *dimension* "`TimeHorizon`": entry `TotalHeatDemand[ t ]` is assumed to contain the total heat demand of this heat block to be satisfied for the corresponding time instant t. Note that this variable does *not* use the "`NumberIntervals`" / "`ChangeIntervals`" system, as demand is usually changing from one time instant to the next;

- the variable "`CostHeatUnit`", of type double and indexed over two *dimensions*. The first *dimension* can have size 1 or "`NumberIntervals`". The second *dimension* has size "`NumberHeatUnits`". This is meant to represent the matrix `CHU[ t , i ]` which is assumed to contain the unitary cost of heat production of heat-producing unit i for time instant t. If the first *dimension* has size 1, then the cost is the same for all time instants (for the same unit). Otherwise, `CostHeatUnit[ h , i ]` is the fixed value of `CHU[ t , i ]` for all t in the interval `[ ChangeIntervals[ h - 1 ], ChangeIntervals[ h ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`;

- the *variable* "`MinHeatProduction`", of type double and indexed over two *dimensions*. The first *dimension* can have size 1 or "`NumberIntervals`". The second *dimension* has size "`NumberHeatUnits`". This is meant to represent the matrix "`MinHP[ t , i ]`" which is assumed to contain the minimum heat production of heat-producing unit i for time instant t. The variable is optional; if it is not provided at all, it is intended "`MnHP[ t , i ] == 0`" for all i and t. If the first dimension has size 1, then the minimum heat production is the same for all time instants (for the same unit). Otherwise, `MinHeatProduction[ h , i ]` is the fixed value of `MinHP[ t , i ]` for all t in the interval `[ ChangeIntervals[ h - 1 ], ChangeIntervals[ h ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`;

- the *variable* "`MaxHeatProduction`", of type double and indexed over two *dimensions*. The first *dimension* can have size 1 or "`NumberIntervals`". The second *dimension* has size "`NumberHeatUnits`". This is meant to represent the matrix "`MaxHP[ t , i ]`" which is assumed to contain the maximum heat production of heat-producing unit i for time instant t. It is assumed `MaxHP[ t , i ] >= MinHP[ t , i ] >= 0` for all i and t, with strict inequality holding for at least some t for each unit i (otherwise the production of unit i is fixed and there is nothing to decide). The variable is optional; if it is not provided at all, it is intended "`MaxHP[ t , i ] == 0`" for all i and t. If the first dimension has size 1, then the maximum heat production is the same for all time instants (for the same unit). Otherwise, `MaxHeatProduction[ h , i ]` is the fixed value of `MaxHP[ t , i ]` for all t in the interval `[ ChangeIntervals[ h - 1 ], ChangeIntervals[ h ] ]`, with the assumption that `ChangeIntervals[ - 1 ] = 0`;

- the *variable* "`MinHeatStorage`", of type double and either indexed over the *dimension* "`NumberIntervals`" or has size 1. This is meant to represent the vector `MinHS[ t ]` which, for each time instant t, contains the minimum heat storage of this HB. The variable is optional, if it is not provided at all it is intended that `MinHS[ t ] == 0` for all t. If the variable has size 1, then the minimum heat storage is the

same for all time instants. Otherwise, `MinHeatStorage[ h ]` is the fixed value of `MinHS[ t ]` for all t in the interval `[ ChangeIntervals[ h - 1 ], ChangeIntervals[ h ] ]`, with the assumption that `ChangeIntervals[ -1 ] = 0`;

- the *variable* "`MaxHeatStorage`", of type double and either indexed over the *dimension* "`NumberIntervals`" or has size 1. This is meant to represent the vector `MaxHS[ t ]` which, for each time instant t, contains the maximum heat storage of this HB. The variable is optional, if it is not provided at all it is intended that `MaxHS[ t ] == 0` for all t and since it's assumed that `MaxHS[ t ] >= MinHS[ t ] >= 0` for all t, this means that there is no heat storage in this HB.. If the variable has size 1, then the maximum heat storage is the same for all time instants. Otherwise, `MaxHeatStorage[ h ]` is the fixed value of `MaxHS[ t ]` for all t in the interval `[ ChangeIntervals[ h - 1 ], ChangeIntervals[ h ] ]`, with the assumption that `ChangeIntervals[ -1 ] = 0`;

- the scalar *variable* "`InitialHeatAvailable`", of type double and not indexed over any *dimension*, which indicates the the amount of heat in the storage at the beginning of the first time instant in this HB. It is assumed that "`MaxHS[ 0 ] >= InitialHeatAvailable >= MinHS[ 0 ]`". This variable is optional, if it is not provided it is taken to be `InitialHeatAvailable == MinHS[ 0 ]`. If there is no heat storage (say, `MaxHeatStorage` is not defined) then this variable is not read, because it is not used;

- the scalar *variable* "`StoringHeatRho`", of type double and not indexed over any *dimension*, which indicates the inefficiency of storing heat in the heat storage (if any) in this HB. This variable is optional and it must always be `StoringHeatRho <= 1`, if it is not provided it is taken to be `StoringHeatRho == 1`. If there is no heat storage (say, MaxHeatStorage is not defined) then this variable is not read, because it is not used;

- the scalar *variable* "`ExtractingHeatRho`", of type double and not indexed over any *dimension*, and which indicates the inefficiency of extracting heat from the heat storage (if any) in this HB. This variable is optional and it must always be `ExtractingHeatRho >= 1`, if it is not provided it is taken to be `ExtractingHeatRho == 1`. If there is no heat storage (say, `MaxHeatStorage` is not defined) then this variable is not read, because it is not used;

- the scalar *variable* "`KeepingHeatRho`", of type double and not indexed over any *dimension*, which indicates the double of keeping heat in the heat storage (if any) in this HB. This variable is optional and it must always be `KeepingHeatRho <= 1`, if it is not provided it is taken to be `KeepingHeatRho == 1`. If there is no heat storage (say, `MaxHeatStorage` is not defined) then this variable is not read, because it is not used;

5 BlockConfig for existing :Block

As described in §3.2.1, each Block can have several Configuration parameters, which can be dealt with by a BlockConfig object. It makes therefore sense to collect here all possible Configuration parameters for existing :Block, since these may end up in a SMS++ NETCDF file.

5.1 Parameters of MCFBlock

The MCFBlock class defines the following Configuration parameters:

- f_static_constraints_Configuration. The static constraint of a MCF are the flow conservation equations and the bound constraints. The latter are typically implemented via a std::vector<LB0Constraint> with exactly m entries. However, If *all* the upper bound are $+\infty$, it is possible to skip the std::vector<LB0Constraint entirely and just use the fact that the ColVariable can be defined to be non-negative. If f_static_constr is set, it is a SimpleConfiguration<int> whose f_value is not 0, the bound constraints are not implemented if it is possible to do so; otherwise they are always implemented. Note that this makes it impossible to change any upper bound.

- f_is_feasible_Configuration. If f_is_feasible_Configuration is a SimpleConfiguration<double>, its f_value is taken as the parameter for deciding what "approximately feasible" exactly means while checking satisfaction of both flow conservation constraint and flow upper/lower bounds; the value is taken as the *relative* tolerance for those checks.

- f_is_optimal_Configuration. Checking optimality of a primal-dual solution of the MCF problems means checking that both are approximately feasible, and that they approximately satisfy the Complementary Slackness Conditions. This requires two parameters for deciding what "approximately feasible" means, one for the primal (ϵ_f) and one for the dual (ϵ_c). If f_is_optimal_Configuration is a SimpleConfiguration< std::pair<double,double> >, then ϵ_c = f_value.first and ϵ_f = f_value.second.

- f_solution_Configuration. For all the methods dealing with solutions, i.e., get_Solution() and map_[back/forw if f_solution_Configuration is a SimpleConfiguration<int>, then its f_value decides what is mapped, with the following encoding: 1 means "only map the primal solution", 2 means "only map the dual solution", everything else (e.g., 0) means " "map everything".

6 ComputeConfig for existing :Solver

As described in §3.2.3, each Block can have several ComputeConfig parameters, which can be dealt with by a BlockSolverConfig object. It makes therefore sense to collect here all possible ComputeConfig parameters for the existing objects that define actual values for their algorithmic parameters, since these may end up in a SMS++ NETCDF file.

6.1 Standard parameters in Solver

The base Solver class defines a set of "general" algorithmic parameters, described in the following Table.

name	title	description
intMaxIter	int	The algorithmic parameter for setting the maximum number of iterations that the next call to solve() is allowed to execute for trying to solve the Block. The concept of "what exactly an iteration is" is clearly Solver-dependent, and the user of the Solver need supposedly be aware of which concrete :Solver it is actually using to be able to sensibly set this parameter; however, because most Solver will actually be iterative processes, it makes sense to offer support for this notion in the base class.
intMaxSol	int	The algorithmic parameter for setting the maximum number of different solutions to the Block that the Solver should attempt to obtain and store. Mathematical models can have (very) many solutions: an objective function precisely helps in selecting among them, but even that may not be enough to narrow the choice down to a single solution (multiple optima may exist, or quasi-optimal solutions may also be sought for for any number of reasons). On the other hand, a solution can be a "big" object, and storing it may be costly. It may be therefore helpful for a Solver to know beforehand how many different solutions the user would like to get. Among the reasonable values for this parameter, 1 says "I don't care of multiple solutions, give me only the best one", and 0 says "I don't care of solutions at all, just tell me if there is any, and what its value is". The default is 1.
intLogVerb	int	An integer parameter dictating how "verbose" the log of the Solver has to be. The specific meaning of each value is Solver-dependent, but it is intended that 0 means "no log at all", and increasing values correspond to increasing verbosity. The default value is 0 (no log).
dblMaxTime	dbl	The algorithmic parameter for setting the maximum time limit that the next call to solve() can expend in trying to solve the Block. The value is assumed to be in seconds, and it's a double (so both very quick and very slow solvers are supported). The base Solver class does not explicitly distinguish between "wall-clock time" and "CPU time", which may be rather different especially in a parallel environment, but this concept can be easily added by derived classes.
dblRelAcc	dbl	The algorithmic parameter for setting the *relative* accuracy required to the solution of the Block. This is geared towards single-objective optimization problems, and it is defined as follows: a solution for a *minimization* problem has a relative accuracy of ε if a feasible (to within the required tolerance for each Block) solution has been obtained, an upper bound ub on its objective function value has been found, a lower bound lb on the optimal value of the problem has been found, and $ub - lb \leq \varepsilon \max(|lb|, 1)$. The roles of ub and lb are suitably reversed for a maximization problem. The default is 1e-6.
dblAbsAcc	dbl	The algorithmic parameter for setting the *absolute* accuracy required to the solution of the model. This is geared towards single-objective optimization problems, and it is defined as follows: a solution for a *minimization* problem has a absolute accuracy of ε if a feasible (to within the required tolerance for each Block) solution has been obtained, an upper bound ub on its objective function value has been found, a lower bound lb on the optimal value of the problem has been found, and $ub - lb \leq \varepsilon$. The roles of ub and lb are suitably reversed for a maximization problem. The default is $+\infty$, which is intended to mean that the only working accuracy is the relative one.

dblUpCutOff	dbl	The algorithmic parameter for setting the *upper cut off* of the solution process. This is a value basically telling the Solver "when enough is enough". In particular, if the Solver obtains a certified lower bound lb on the optimal value such that $lb \geq \varepsilon$ with the provided parameter ε, then it can stop. This is a certificate that the optimal value is at least ε. For a maximization problem the condition says: I actually needed a solution with objective function value at least as good as (i.e., larger than) ε, now that I have found one the problem is as good as solved to me. Hence this parameter is analogous to the maximum absolute accuracy dblAbsAcc, but "weaker" in that it does not not require any upper bound to work. For a minimization problem, instead, the condition says: I actually needed a solution with objective function value at least as good as (i.e., smaller than) ε, now that I have know for sure that this is never going to happen the problem is as good as unfeasible to me. The default is $+\infty$.
dblLwCutOff	dbl	The algorithmic parameter for setting the *lower cut off* of the solution process. This is a value basically telling the Solver "when enough is enough". In particular, if the Solver obtains a certified upper bound ub on the optimal value such that $ub \leq \varepsilon$ with the provided parameter ε, then it can stop. This is a certificate that the optimal value is at most ε. For a minimization problem the condition says: I actually needed a solution with objective function value at least as good as (i.e., smaller than) ε, now that I have found one the problem is as good as solved to me. Hence this parameter is analogous to the maximum absolute accuracy dblAbsAcc, but "weaker" in that it does not not require any lower bound to work. For a maximization problem, instead, the condition says: I actually needed a solution with objective function value at least as good as (i.e., larger than) ε, now that I have know for sure that this is never going to happen the problem is as good as unfeasible to me. The default is $-\infty$.
dblRAccSol	dbl	The algorithmic parameter for setting the relative accuracy of the accepted solutions. It instructs the Solver not to even consider a solution among the ones to be reported intMaxSol if its objective function value is "too" bad. For a minimization problem, the objective function value of a feasible solution provides an upper bound ub on the optimal value. Assuming a lower bound lb on the optimal value of the problem has been found, a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon \max(\lvert ub\rvert, \lvert lb\rvert, 1)$. Note that if no lb is available, the above formula can be replaced with $ub - f_{best} \leq \varepsilon \max(\lvert ub\rvert, \lvert f_{best}\rvert, 1)$, where f_{best} is the value of the best (with smallest objective value) solution found so far. The roles of ub and lb are suitably reversed for a maximization problem. The default is $+\infty$.
dblAAccSol	dbl	Similar to dblRAccSol but for an *absolute* accuracy; that is, a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon$ or $ub - f_{best} \leq \varepsilon$ with the same notation as in dblRAccSol and the same provisions about the case of a maximization problem. The default is $+\infty$.
dblFAccSol	dbl	The algorithmic parameter for setting the maximum relative allowed violation of constraints. Whenever the Solver is incapable of finding feasible solutions (maybe because there is none), it may still be useful that it returns the "least unfeasible" ones. This parameter instructs the Solver not to even consider a solution among the ones to be reported (see intMaxSol) if its violation is "too" bad. The actual meaning of this parameter may be Solver-dependent, but it can be thought to work as the "relative constraint violation". A setting of 0 may be taken as a way to tell the Solver not to bother to produce unfeasible solutions at all, which is why this is the default value of the parameter.

6.2 Standard parameters in CDASolver

The CDASolver class extends the parameters of Solver with the ones described in the following Table.

name	title	description

intMaxDSol	int	The algorithmic parameter for setting the maximum number of different *dual* solutions that the Solver should attempt to obtain and store. Since dual solutions can be a "big" objects (in some cases, much bigger than primal sones), storing them may be costly. It may be therefore helpful for a CDASolver to know beforehand how many different dual solutions the user would like to get. Among the reasonable values for this parameter, 1 says "I don't care of multuple dual solutions, give me only the best one", and 0 says "I don't care of dual solutions at all, just tell me if there is any, and what its value is". The default is 1.
dblRAccDSol	dbl	The algorithmic parameter for setting the relative accuracy of the accepted *dual* solutions. It instructs the CDASolver not to even consider a solution among the ones to be reported (see intMaxDSol) if its objective function value is "too" bad. If the original problem is a minimization one, its dual is a maximization one. Hence, the objective function value of a dual solution provides a lower bound lb on the optimal dual value (which may, or may not, be equal to that of the primal problem). Assuming an upper bound ub on the optimal value of the *dual* problem has been found (note that this can be obtained by means of the objective value of a feasible *primal* solution), a solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon \max(|ub|, |lb|, 1)$. Note that if no ub is available, the above formula can be replaced with $f_{best} - lb \leq \varepsilon \max(|f_{best}|, |lb|, 1)$ where f_{best} is the value of the best (with largest objective value) solution found so far. The roles of ub and lb are suitably reversed if the primal is a maximization problem, so that the dual is a minimization one. The default is $+\infty$.
dblAAccDSol	dbl	Similar to dblRAccDSol but for an *absolute* accuracy; that is, a dual solution is deemed acceptable with the provided parameter ε if $ub - lb \leq \varepsilon$ or $f_{best} - lb \leq \varepsilon$ with the same notation as in dblRAccDSol and the same provisions about the case of a maximization problem. The default is $+\infty$.
dblFAccDSol	dbl	The algorithmic parameter for setting the maximum relative allowed violation of *dual* constraints, assuming of course something like that exists in the specific dual that is being dealt with. Whenever the CDASolver is incapable of finding feasible dual solutions (maybe because there is none), it may still be useful that it returns the "least unfeasible" ones. This parameter instructs the CDASolver not to even consider a solution among the ones to be reported (see intMaxDSol) if its violation is "too" bad. The actual meaning of this parameter is necessarily be CDASolver-dependent; intuitively, it may be thought to work as the "relative constraint violation" feas_epsilon of Block if the concept of "dual constraint" is applicable. A setting of 0 may be taken as a way to tell the CDASolver not to bother to produce unfeasible solutions at all, which is why this is the default value of the parameter.

6.3 Standard parameters in Function

The base Function class defines a set of "general" algorithmic parameters, described in the following Table.

name	title	description
intMaxIter	int	The algorithmic parameter for setting the maximum number of iterations in the next call to compute(). The concept of "what exactly an iteration is" is clearly dependent on the Function, and it may not even make sense for all Function. However, some Function will actually be iterative processes, and therefore it makes sense to offer support for this notion in the base class. The default is $+\infty$ = no limits.
dblMaxTime	dbl	The parameter for setting the maximum time limit for the next call to compute(). The value is assumed to be in seconds, and it's a double (so both very fast and very slow computations are supported). The Function class (so far) does not explicitly distinguish between "wall-clock time" and "CPU time", which may be rather different especially in a parallel environment. The default is $+\infty$.
dblRelAcc	dbl	The parameter for setting the *relative* accuracy required to the function value. That is, if both an upper bound ub and a lower bound lb on the value have been found, then compute() can stop as soon as $ub - lb \leq$ dblRelAcc $\max(|lb|, 1)$. The default is 1e-6.

dblAbsAcc	dbl	The parameter for setting the *absolute* accuracy required to the function value. That is, if both an upper bound ub and a lower bound lb on the value have been found, then compute() can stop as soon as $ub - lb \leq$ **dblRelAcc**. The default is $+\infty$, which is intended to mean that the only working accuracy is the relative one.
dblUpCutOff	dbl	The parameter for setting the "upper cut off" of the computation; that is, if a lower bound lb on the value has been found, then compute() can stop as soon as $lb \geq$ **dblUpCutOff**. This is a certificate that the value is at least dblUpCutOff. The default is $+\infty$, i.e., no upper cut off.
dblLwCutOff	dbl	The parameter for setting the "lower cut off" of the computation; that is, if an upper bound ub on the value has been found, then compute() can stop as soon as $ub \leq$ **dblLwCutOff**. This is a certificate that the value is at most dblLwCutOff. The default is $-\infty$, i.e., no lower cut off.

6.4 Standard parameters in C05Function

The C05Function class extends the parameters of Function with the ones described in the following Table.

name	title	description
intLPMaxSz	int	The algorithmic parameter for setting the size of the "local pool", that is, the maximum number of linearizations that should be stored in the local pool. The default is 1, which corresponds to the fact that the Function can only produce a single linearization at a time (for it is, say, smooth).
intGPMaxSz	int	The algorithmic parameter for setting the size of the "global pool", that is, the maximum number of linearizations that should be stored in the local pool. The default is 0, which corresponds to the fact that the Function cannot store any linearization (for it is, say, smooth and therefore there is no need to).
dblRAccLin	dbl	A linearization (g, α) computed at the point x is "accurate" if the value of the linearization coincides with the value of the function at x, i.e., $\alpha = f(x)$. In general linearizations that are not "completely accurate" can still be useful: for instance, in the Lagrangian case an ε-optimal solution to the Lagrangian problem gives rise to a valid linearization (g, α) with $\varepsilon \geq f(x) - \alpha$. This can be deemed interesting if ε is "small", but not if ε is "large". This parameter instructs the C05Function not to bother reporting (and therefore storing in the "local pool") any linearization having a relative error with $f(x)$ larger than dblRAccLin. This would generally mean $\lvert f(x) - \alpha \rvert \leq$ **dblRAccLin** $\max(\lvert f(x) \rvert, \lvert \alpha \rvert, 1)$, except that the value $f(x)$ may not be known exactly, with only lower and/or upper bounds on it available. The actual formula therefore depends on what information is actually available: for instance, in the Lagrangian case one knows that $f(x) \geq \alpha$, and therefore typically an upper estimate $ub \geq f(x)$ is used in the formula instead of $f(x)$. The default is 0, i.e., "only perfect linearizations are allowed".
dblAAccLin	dbl	Similar to dblRAccLin but for an *absolute* accuracy; that is, a linearization is deemed acceptable if $\lvert f(x) - \alpha \rvert \leq$ **dblAAccLin**, except that the value $f(x)$ may not be known exactly, with only lower and/or upper bounds on it available. The actual formula therefore depends on what information is actually available: for instance, in the Lagrangian case one knows that $f(x) \geq \alpha$, and therefore typically an upper estimate $ub \geq f(x)$ is used in the formula instead of $f(x)$. The default is 0, i.e., "only perfect linearizations are allowed".

6.5 Parameters of MCFSolver<MCFC>

The MCFSolver<> class derives by CDASolver, and therefore it extends the parameters of with the ones described in the following Table.

name	title	description
kReopt	int	Whether the MCF algorithm (whatever it is) should re-optimize after changes in the data, as opposed to restarting from scratch.

However, `MCFSolver<>` is a template class, whose template parameter actually is a concrete MCF solver object derived from `MCFClass`. Each of these has its own algorithmic parameters, which are described in the next subsections.

6.5.1 Parameters of `MCFSolver<MCFSimplex>`

name	title	description
`kAlgPrimal`	`int`	If > 0 it instructs the `MCFSimplex` solver to use the primal (network) simplex method, otherwise the dual one.
`kAlgPricing`	`int`	Selects which pricing rule is used within the algorithm. Possible values are `kDantzig` $= 0$ (Dantzig's rule, most violated constraint), `kFirstEligibleArc` $= 1$ (first eligible arc in round-robin), and `kCandidateListPivot` $= 2$ (Candidate List Pivot Rule).
`kNumCandList`	`int`	Parameter to set the number of candidate list for the Candidate List Pivot method.
`kHotListSize`	`int`	Parameter to set the size of Hot List for the Candidate List Pivot method.

7 Conclusion

This document is intended as a quick reference manual for anybody who is required to construct SMS++ NETCDF input files. It should be remarked that basically all the `:Block` and `:Configuration` objects are supposed to have an in-memory interface to initialize the data. Thus, a workable way to produce NETCDF input files for those is to use the in-memory interface, and then the `serialize()` method to save it into a file. Yet, NETCDF has interfaces for all primary programming languages, hence files with the required format can be written independently by any SMS++ object.

References

[1] Unidata "The NETCDF software", http://doi.org/10.5065/D6H70CW6, 2019

3